CARRERA: INGENIERÍA EN SISTEMAS COMPUTACIONALES

GRUPO: 3° "B"

MÉTODOS NÚMERICOS

DOCENTE: VÍCTOR HUGO VÁSQUEZ HERRERA

Reporte Tarea 1 Unidad 4: Simpson 1/3 Compuesto vs Trapecio Compuesto

ALUMNO: Mejía Mora Brayan

MATRÍCULA: 257O0081

SÁBADO 24 DE OCTUBRE 2026

```
for (int k = 1; k < n; k++) {
        double xk = a + k * h;

        // si k es impar, lleva coeficiente 4; si k es par, lleva coeficiente 2
        if (k % 2 != 0) {
                sumaImpares += TrapecioCompuesto.funcion(xk);
        } else {
                sumaPares += TrapecioCompuesto.funcion(xk);
        }
}

double sumaTotal = TrapecioCompuesto.funcion(a) + 4 * sumaImpares
                + 2 * sumaPares + TrapecioCompuesto.funcion(b);

return (h / 3) * sumaTotal;
```

La condición `k % 2 != 0` detecta los índices impares (x_1, x_3, x_5, ...), que son los puntos medios de cada parábola. Los índices pares interiores (x_2, x_4, ...) son los puntos que dos parábolas vecinas comparten. Los extremos f(a) y f(b) se suman una sola vez, fuera del ciclo.

La clase reutiliza `calcularH` y `funcion` de `TrapecioCompuesto`, así que ambos métodos integran exactamente la misma función. En `integrarConTolerancia`, n inicia en 2 y se duplica en cada vuelta (2, 4, 8, ...), por lo que siempre es par y la regla de Simpson 1/3 es aplicable en todas las iteraciones.

Resultados comparativos

Con $f(x) = x^2$ en [0, 2] y tolerancia 0.0001, el Trapecio Compuesto necesita 7 iteraciones y n = 256, mientras que Simpson 1/3 Compuesto converge en 1 iteración con n = 4.

Se cuenta una iteración por cada vuelta del `while` (cada duplicación de n). El valor n = 2 es la aproximación inicial y no cuenta como iteración.

Métrica	Trapecio Compuesto	Simpson 1/3 Compuesto
n inicial	2	2
Iteraciones del `while`	7	1
n final (subintervalos)	256	4
Aproximaciones calculadas (incluye la inicial)	8	2

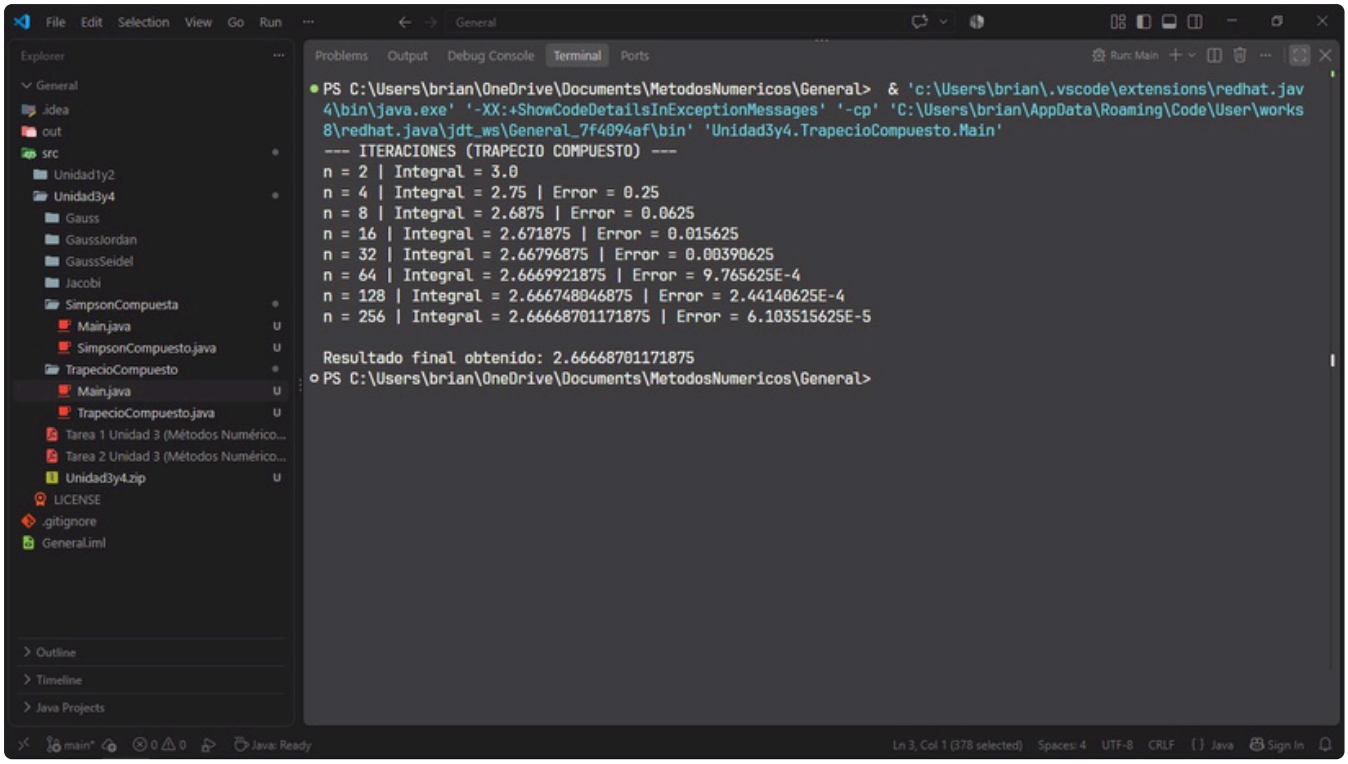
Métrica	Trapecio Compuesto	Simpson 1/3 Compuesto
Evaluaciones acumuladas de f(x)	518	8
Resultado final	2.66668701171875	2.6666666666666665
Último error entre iteraciones	6.1035×10^{-5}	0.0
Error real respecto a 8/3	$\approx 2.03 \times 10^{-5}$	$\approx 10^{-16}$ (límite de precisión de `double`)

Las evaluaciones de f(x) se cuentan como n + 1 por aproximación, porque cada una se recalcula desde cero: 3 + 5 + 9 + 17 + 33 + 65 + 129 + 257 = 518 para Trapecio y 3 + 5 = 8 para Simpson.

Salida en consola: Trapecio Compuesto

```
--- ITERACIONES (TRAPECIO COMPUESTO) ---
n = 2 | Integral = 3.0
n = 4 | Integral = 2.75 | Error = 0.25
n = 8 | Integral = 2.6875 | Error = 0.0625
n = 16 | Integral = 2.671875 | Error = 0.015625
n = 32 | Integral = 2.66796875 | Error = 0.00390625
n = 64 | Integral = 2.6669921875 | Error = 9.765625E-4
n = 128 | Integral = 2.666748046875 | Error = 2.44140625E-4
n = 256 | Integral = 2.66668701171875 | Error = 6.103515625E-5

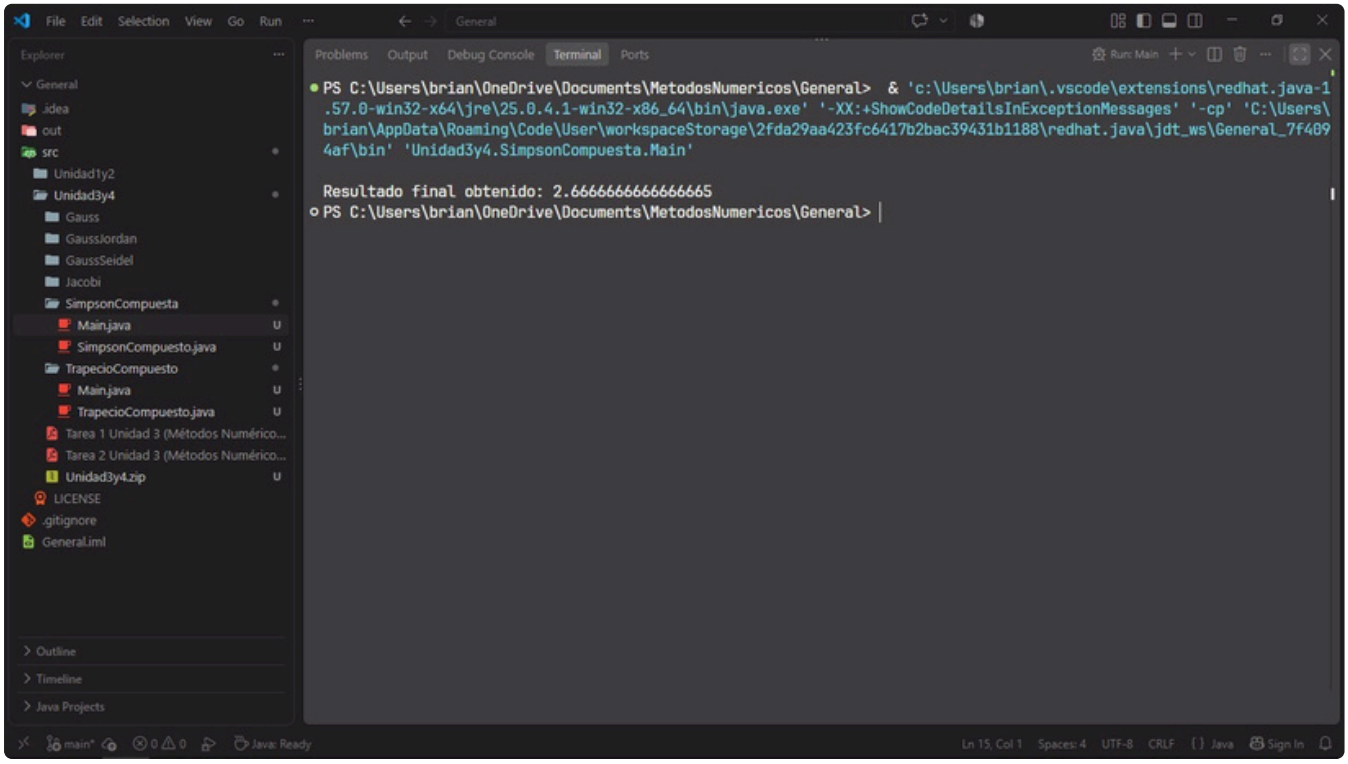
Resultado final obtenido: 2.66668701171875
```



Salida en consola: Simpson 1/3 Compuesto

--- ITERACIONES (SIMPSON COMPUESTO) ---
n = 2 | Integral = 2.6666666666666665
n = 4 | Integral = 2.6666666666666665 | Error = 0.0

Resultado final obtenido: 2.6666666666666665



Interpretación

El Trapecio reduce su error a la cuarta parte cada vez que n se duplica (0.25, 0.0625, 0.015625, …), es decir, converge con orden h^2. Con n = 128 la diferencia entre iteraciones (2.44×10^{-4}) aún supera la tolerancia; con n = 256 baja a 6.10×10^{-5} y el ciclo termina. Simpson converge de inmediato porque su error depende de la cuarta derivada de f, y para $f(x) = x^2$ esa derivada vale cero: la regla es exacta para polinomios de grado 3 o menor. Por eso n = 2 y n = 4 dan el mismo valor y el error entre ambas es 0.0. Con una función no polinómica, Simpson también necesitaría más iteraciones, pero muchas menos que el Trapecio por su orden h^4.

Análisis conceptual

¿Por qué n debe ser estrictamente par?

Simpson 1/3 ajusta una parábola a cada grupo de tres puntos consecutivos, y esos tres puntos abarcan dos subintervalos; por eso los n subintervalos deben poder agruparse en parejas.

> Una parábola queda determinada por tres puntos: la primera usa x_0, x_1, x_2; la segunda, x_2, x_3, x_4; y así sucesivamente.
> Cada parábola cubre un tramo de ancho 2h (dos subintervalos) y su área es:

$$\int_{x_{2i}}^{x_{2i+2}} f(x)\,dx \approx \frac{h}{3}\left[\, f(x_{2i}) + 4f(x_{2i+1}) + f(x_{2i+2}) \,\right]$$

- Para cubrir todo [a, b] sin huecos ni traslapes hacen falta n/2 parábolas, y n/2 solo es entero si n es par.
- Si n fuera impar, quedaría un subintervalo suelto con solo dos puntos. Por dos puntos pasan infinitas parábolas, así que la regla no se puede aplicar y ese tramo exigiría otro método (por ejemplo, un trapecio o Simpson 3/8).

En el código, n inicia en 2 y se duplica en cada vuelta (2, 4, 8, …), por lo que siempre es par.

¿De dónde sale el patrón 1, 4, 2, 4, …, 1?

Cada coeficiente cuenta cuántas veces aparece un punto en la suma de las áreas de las figuras: un punto compartido por dos figuras suma su aporte dos veces.

Trapecio (figura de dos puntos). Cada trapecio cubre $[x_k, x_{k+1}]$ y aporta $(h/2)\cdot[f(x_k) + f(x_{k+1})]$. Todo punto interior es el extremo derecho de un trapecio y el izquierdo del siguiente, así que aparece dos veces: coeficiente 2. Los extremos x_0 y x_n pertenecen a un solo trapecio: coeficiente 1. Resultado: 1, 2, 2, …, 2, 1.

Simpson (figura de tres puntos). Cada parábola aporta (h/3)·[1·f(izquierdo) + 4·f(medio) + 1·f(derecho)]. Los índices impares (x_1, x_3, …) son el punto medio de su parábola y no se comparten: coeficiente 4. Los índices pares interiores (x_2, x_4, …) son el extremo derecho de una parábola y el izquierdo de la siguiente: 1 + 1 = 2. Los extremos x_0 y x_n pertenecen a una sola parábola: coeficiente 1. Como se alternan puntos medios y puntos compartidos, el patrón es 1, 4, 2, 4, 2, …, 4, 1.

Ejemplo con n = 8:

Punto	x_0	x_1	x_2	x_3	x_4	x_5	x_6	x_7	x_8
Coeficiente Trapecio (factor h/2)	1	2	2	2	2	2	2	2	1
Parábolas que contienen el punto	1	1	2	1	2	1	2	1	1
Coeficiente Simpson (factor h/3)	1	4	2	4	2	4	2	4	1

Comprobación: si f(x) = 1, la suma de coeficientes da 16 · (h/2) = 8h para el Trapecio y 24 · (h/3) = 8h para Simpson, y en ambos casos 8h = n·h = b − a, el área correcta. En el código, `k % 2 != 0` (coeficiente 4) y `else` (coeficiente 2) reproducen exactamente esta tabla.

Conclusiones

Para $f(x) = x^2$ en [0, 2] con tolerancia 0.0001, Simpson 1/3 Compuesto es mucho más eficiente que el Trapecio Compuesto: llega al resultado con 8 evaluaciones de f(x) frente a 518, casi 65 veces menos.

- **Eficiencia:** Simpson converge en 1 iteración (n = 4) y el Trapecio en 7 (n = 256).
- **Precisión:** el error real de Simpson es del orden de 10^{-16} y el del Trapecio, de 2.03×10^{-5}. **Causa:** el error de Simpson es de orden h^4 y depende de la cuarta derivada, que es cero para x^2; el del Trapecio es de orden h^2 y se reduce a la cuarta parte en cada duplicación de n.
- **Condición de uso:** Simpson 1/3 exige n par porque cada parábola abarca dos subintervalos. El código lo garantiza al iniciar en n = 2 y duplicar.
- **Alcance:** x^2 es un caso especialmente favorable para Simpson, por ser un polinomio de grado 2. Este reporte no probó otras funciones, donde Simpson también requeriría más iteraciones.

Repositorio de GitHub

https://github.com/brianlewyn/numerical-methods-tec-3th-semester/tree/main/Unidad3y4

El repositorio contiene el proyecto completo en Java y un `README.md` en la raíz con las instrucciones para compilar y ejecutar desde la terminal o el IDE, y con un ejemplo de la salida en consola de ambos métodos.

Archivo	Contenido
src/Unidad3y4/TrapecioCompuesto/TrapecioCompuesto.java	f(x), cálculo de h, Trapecio Compuesto e iteración con tolerancia
src/Unidad3y4/TrapecioCompuesto/Main.java	Ejecuta el Trapecio con a = 0, b = 2, tolerancia = 0.0001
src/Unidad3y4/SimpsonCompuesta/SimpsonCompuesto.java	Simpson 1/3 Compuesto con el // TODO completado e iteración con tolerancia
src/Unidad3y4/SimpsonCompuesta/Main.java	Ejecuta Simpson con los mismos datos
README.md	Instrucciones de compilación y ejecución, y ejemplo de salida